

Communication Support in a Process-Centered Software Engineering Environment

Volker Gruhn
LION GmbH
Universitätsstraße 140
D-44799 Bochum
Federal Republic of Germany
gruhn@alf.lion.de

Abstract

This position paper distinguishes different types of activities needed in software processes and proposes enactment support for them. An architecture of enactment components providing this support is sketched.

1 Human participation in software processes

Most of today's software developments can only be accomplished in a joint effort by many software developers: in some projects, work may be divided among ten of them, in others among hundreds, and in some large ones even among thousands of developers. The management of people alone, their allocation to tasks according to their skills, the communication between them, the integration of their respective results represents a task of formidable complexity.

In fact one may think of collaborative software development as a distributed activity with many obstacles that must be overcome to ensure the appropriate communication among not necessarily geographically but "conceptually distributed" software developers since they experience all the difficulties that characterize human communication.

A process-centered software engineering environment should allow the comfortable modeling of human involvement in software processes. Moreover, it should provide communication support for the different types of communication which build software processes.

2 Different types of activities in software processes

The very most explanations of the term process (in the software process community, in the industrial engineering community [2] or in the business process community [6]) center around the notion of *activities* [5]. An *activity* is usually understood as a unit of work that cannot be usefully subdivided. This is obviously not a formal definition. It depends on who decides about activity subdivision. Accordingly, the notion of *activity* cannot be formalized.

Nonetheless activities build the skeleton of processes. They are the entities where human involvement into software processes can be pinpointed. Activities may require different kinds of human participation [10]. In order to discuss mechanisms needed to support human actors, we distinguish different kinds of activities.

Automatic activities: Automatic activities can be carried out without any human intervention. They can be carried out by devices. Examples are:

- compiling a module as soon as it is edited,
- archiving a document as soon as it is released,
- producing a back up of a system state at a predefined point of time.

Automatic activities have to be programmed. It has to be defined which piece of software has to be executed, when the activity is started. It has to be defined how parameters are passed to that activity and where results are stored. Thus, they are described by an input/output behavior and by a body, which specifies what has to happen, when the activity is executed (compare below).

```
archive:  IN f1:  file;
          INOUT path: file-sys;
BODY:    copy (f1, file-sys);
DESCR:   archive a file in a certain
          directory of a file system.
```

Individual interaction activities: Individual interaction activities require the participation of only one human being. They can be carried out either by a human actor and a device or by a device alone. Examples are:

- editing a module,
- reading a document.

In addition to a description which is similar to the description of an automatic activity (input/output behavior and body), an individual interaction activity contains an informal description. This informal description is a help for the human actor who participates. Moreover, it is defined which human actors are allowed to participate. This can either be done by identifying the potential participants by name, or it can be done via a role concept [8]. Besides the general definition of roles, it is moreover necessary to define, which roles/persons are allowed to access which documents. It may, for example, be useful to define that everybody playing the role *software developer* is allowed to edit modules, but that everybody who modifies the database interface modules must have the additional qualification of being SQL expert. Thus, permissions to participate in activities are defined by roles and value-dependent restrictions. The example discussed in the following is an *edit*-activity, in which the tool *emacs* is used. It is defined that human actors playing the role *software developer* are allowed to participate, and that moreover only software developers to whom the corresponding module is assigned are allowed to edit it. The additional constraint specifies that database modules can be modified only by software developers with the qualification of SQL experts.

```
edit:  INOUT m1: module;
      emacs (m1);
DESCR:  edit a module with emacs in
        write mode
ROLE:   software developer
CONSTRAINT: m1 must be assigned to
            software developer;
            if m1 ∈ {db-modules} :
              QUAL: SQL-expert;
```

Social interaction activities: Social

interaction activities require the interaction of several human beings. They are carried out by two or more human actors.

Examples are:

- discussing a design document in a review team,
- report results in a meeting.

We distinguish these activities from individual interaction activities, because *group activity is vastly different from individual activity and has its own needs and requirements* [7]. Section 4 will show that different enaction mechanisms are required for individual and social interaction activities.

Social interaction activities are described by their input/output behavior, by their body, and

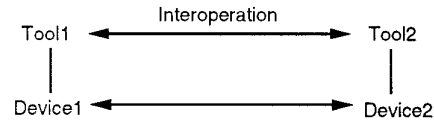


Figure 1: Interoperation communication

by a so-called dialogue description. A dialogue description defines how many persons playing which roles have to interact with each other to carry out the described activity. Moreover, they define which of the interacting persons is supposed to be the interaction manager (compare section 4).

We consider this distinction of activity types useful because they require different kinds of communication support.

3 Communication support

Different kinds of activities demand different kinds of communication support [1]. The communication between two automatic activities is different from the communication between an automatic activity and an individual interaction activity, which is itself different from communication support needed between human actors participating in two related individual interaction activities or even between human actors participating in one social interaction activity.

Communication provided in a completely automatic process (no interaction activities at all) is called **interoperation**. It is closely related to the notion of interoperability [11]. Interoperation support is needed when information is exchanged between automatic activities. Figure 1 shows that interoperation takes place, for instance, between tools that reside on different devices. This communication does not need any human intervention. Examples of interoperation communication are:

- A set of compiled modules must be linked. Therefore, the compiled modules must be communicated between the compiler and the linker.
- A figure produced by a drawing program must be incorporated into a text. Thus, the figure representation must be communicated between the text processing system and the display system.

The communication infrastructure required to support an interaction process (a process in which only one human actor participates) includes additional **interaction** support. It requires communication between a human actor on the one hand, and a tool, respectively a device, on the other hand. Figure 2 shows where interaction occurs in an interaction process. Each of the tools shown in Figure 2 is used by only one human actor. The exchange of information

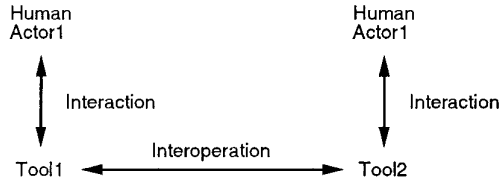


Figure 2: Interoperation and interaction communication

produced by using the tools occurs through interoperation between these tools.

An interpersonal process (a process in which several human actors participate) requires support for communication between human actors. If the communication between different human actors is provided by a chain consisting of an individual interaction activity followed by another individual interaction activity, we call the respective communication **interworking** [1]. This type of interworking is close to the notion of interworking as it is identified in the ESF project [9]. Interworking support is also needed in each social interaction activity. Interworking support is, for instance, needed in the discussion of a document as well as in the joint preparation of a meal. It is needed between a human actor writing a requirements document and another one who transforms that document into a design document.

Figure 3 illustrates the communication infrastructure in support of interoperation, interaction, and interworking [1]. Figure 3 shows two interworking links. One of the interworking links is a direct link between the two human actors. This link depicts non-formalized communication that is not supported by any kind of communication protocol. This kind of interworking communication is based on spoken language. The other interworking link represents the formalized communication between human actors who participate in related individual interaction activities by means of interaction and interoperation support. This kind of interworking communication is indirect.

4 Enaction mechanisms

Interoperation, interaction, and interworking (direct and indirect) require different kinds of enaction support. Enaction support for interoperation requires that automatic activities are started and that data between automatic activities is transported. As soon as all inputs are available and all other activity's preconditions are fulfilled automatic activities can be started.

The enaction mechanism for individual interaction activities is more sophisticated. As soon as all input parameters are available and all other preconditions are fulfilled an individual interaction activity has to be offered to all participants who have the right to par-

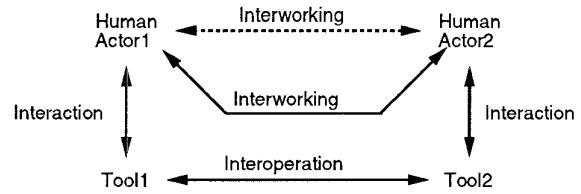


Figure 3: Interoperation, interaction, and interworking communication

ticipate in that activity. Offering an activity to somebody could, for example, be implemented by writing that activity and the actual input parameters into the personal agenda of the process participant. As soon as one of the participants declares that he is ready and willing to participate, the offer has to be removed from the agendas of all other process participants.

The enaction mechanism for social interaction activities is even more difficult. The responsible interaction manager can propose a date for the social interaction. Moreover, the activity's inputs are made accessible to all potential participants. In the example discussed above the meeting's agenda is sent to all potential participants. Based on the agenda items they can decide whether or not they want to participate. Every potential attendee can agree or disagree to the proposed date within a certain period of time (also fixed by the interaction manager). As soon as this period of time is over, the interaction manager checks whether the interaction activity can take place on the proposed date. It can only be scheduled if a sufficient number of attendees accept. What is sufficient in this context is determined by the description of the social interaction activity which should encompass a description of the number and roles of expected participants. If the number of process participants who agree is sufficient, then the interaction manager can decide whether he wants to schedule the meeting for the proposed date or whether he prefers to make another attempt to propose a date. Another attempt may be useful (even if *enough* human actors are ready to participate) if the interaction manager wants particular process participants to participate, and if these are unable to participate in the activity at the date proposed. Once a date has been fixed, all process participants who agreed to that date get an entry in their agenda, which denotes the social interaction activity and the proposed date.

5 Enaction components

The different kinds of enaction mechanisms discussed above are implemented in different enaction components. In the following we discuss an architecture of enaction components. This architecture is implemented in the process management environment

LEU¹ [4].

Figure 4 shows the relationship between different enaction components and their embedding into the overall architecture. Boxes represent components (i.e. module hierarchies). Arrows between boxes indicate that there is a *call*-relationship between these components. The annotations of arrows give examples of demanded services. The component *Process engine*, for example, calls the service *start activity* from component *Automatic activity handler*. Between some components we find double-headed arrows (e.g. between *Process engine* and *Agenda controller*). In that case the arrow annotations indicate which service is demanded by which component (e.g. the *Process engine* demands to *fill entries* into agendas, and the *Agenda controller* returns entries which have been selected (service *selected entry*)). The main functionality of these components and their interfaces to other components are discussed in the following:

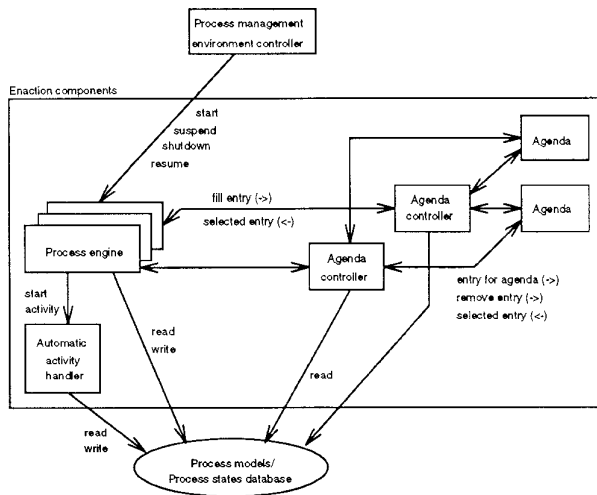


Figure 4: Potential architecture of enaction components

- There is one **Process engine** for each software process running. It can be useful to understand a large project as a set of communicating software processes in order to keep them manageable. Therefore, more than one process engine may be necessary. The process engine drives the process forward. Process engines can be started, suspended, shut down, and resumed by the environment control. The process engine for a process *P* identifies all activities of *P* for which all inputs are available and which, therefore, could be executed. In case of automatic

¹LEU is a process management environment which supports modeling, analysis, and enaction of business processes based on extended entity/relationship diagrams, FUNSOFT nets [3], and organization models.

activities, it starts them via the *Automatic activity handler* (compare below). In case of interaction activities, corresponding entries are sent to the process engine's *Activity controller*. To identify executable activities, a process engine reads the database storing process models and process states. To update the process state after executing an activity, the process state database is modified.

- The only functionality of the **Automatic activity handler** is to receive execution requests from all process engines, to start the requested activities, and to return whether or not the execution has been successful.
- As soon as a process engine is started, a related **Agenda controller** is created. An agenda controller administrates all interaction activities of the related process. It is connected to the agendas of all persons who may participate in the process. The set of persons who may participate can be identified on the basis of roles and permissions which are also stored in the underlying database. Agenda controllers manage individual and social interaction activities. If an individual interaction activity could be executed, the agenda controller sends it to the agendas of all process participants who could participate. If an individual interaction activity is selected by a process participant, the agenda controller returns this information to its process engine which starts the activity at the workstation of the participant. When a social interaction activity could be executed, the agenda controller sends it to all concerned agendas. Then, the agenda controller manages the agreement procedure. When the social interaction activity is finished, the agenda controller returns a corresponding acknowledgement to the process engine.
- There is one **Agenda** for each human actor. As soon as a human actor logs into the process management environment, an agenda is started and automatically connected to the agenda controllers of all processes, the human actor has permissions/roles to participate in. Figure 5 shows an example of an agenda. It contains some status information about the person and his/her role. In the example, *Smith* plays the role *software developer* and is allowed to participate in two processes (*design-database*, *validate-interface*) in which he can interact within three individual interaction (*comment_on_doc*, *suggest_improvements*, *snapshot-ui*) and one social interaction activity (*design-audit*). The last agenda column shows since when the activities are executable. The additional date for the social interaction activity indicates for which point of time the *meeting* is scheduled. The * means, that *Smith* is the responsible interaction manager.

Agenda for Smith			
PROCESS	ACTIVITY	DATE	
design-database			
	individual	comment_on_doc	02 Feb 1994 10:00
		suggested_improvements	02 Feb 1994 11:00
	social	design_audit *	03 Feb 1994 10:43
			10 Feb 1994 14:00
validate-interface			
	individual	snapshot_ui	25 Jan 1994 17:25
	social		
Role: software developer		Logged in: 10:37	

Figure 5: Example of an agenda

6 Conclusion

We have identified different kinds of human involvement into software process activities. We discussed different enaction mechanisms, and we proposed an architecture for corresponding enaction components. The proposed architecture is kept as general as possible. It is independent from any process modeling language, but only based on the assumption that different types of human involvement in activities require communication and enaction support. Our future efforts will be focused on a detailed investigation how the identified kinds of human involvement are reflected by existing groupware, workflow and software process management environments.

References

- [1] R. Adomeit, W. Deiters, B. Holtkamp, R. Rockwell, F. Schülke, and H. Weber. *Software Engineering Environments and Software Factories*. John Wiley and Sons, New York, N.Y., 1992.
- [2] T.H. Davenport and J.E. Short. *The New Industrial Engineering: Information Technology and Business Process Redesign*. Sloan Management Review, 1990.
- [3] W. Deiters and V. Gruhn. *Managing Software Processes in MELMAC*. In *Proceedings of the Fourth ACM SIGSOFT Symposium on Software Development Environments*, pages 193–205, Irvine, California, USA, December 1990.
- [4] G. Dinkhoff, V. Gruhn, A. Saalman, and M. Zielonka. *Business Process Modeling in the Workflow Management Environment LEU*. In P. Loucopoulos, editor, *Proceedings of the 13th International Conference on the Entity-Relationship Approach*, pages 46–63, Manchester, UK, December 1994. Springer. Appeared as Lecture Notes in Computer Science no. 881.
- [5] V. Gruhn. *Software Process Management and Business Process (Re)Engineering*. In B. Warboys, editor, *Software Process Technology - Proceedings of the 3rd European Software Process Modeling Workshop*, pages 250–254, Villard de Lans, France, February 1994. Springer. Appeared as Lecture Notes in Computer Science 772.
- [6] M. Hammer and J. Champy. *Reengineering the Corporation*. Harper Business, New York, US, 1993.
- [7] D.A. Norman. *Collaborative Computing: Collaboration First, Computing Second*. *Communications of the ACM*, 34(12), December 1991.
- [8] J. Sa and B. Warboys. *Integrating a Formal Specification Method with PML: A Case Study*. In J.-C. Derniame, editor, *Software Process Technology - Proceedings of the 2nd European Software Process Modeling Workshop*, pages 106–122, Trondheim, Norway, September 1992. Springer. Appeared as Lecture Notes in Computer Science 635.
- [9] W. Schäfer and H. Weber. *The ESF-Profile*. In *Handbook of Computer Aided Software Engineering*, New York, 1988. Van Nostrand.
- [10] B. Warboys, editor. *Proceedings of the 3rd European Workshop on Software Process Modelling*, Villard de Lans, France, February 1994. Springer. Appeared as Lecture Notes in Computer Science 772.
- [11] J.C. Wileden, A.L. Wolf, W.R. Rosenblatt, and P.L. Tarr. *Specification-Level Interoperability*. *Communications of the ACM*, 34(5), May 1991.